

SIMATS ENGINEERING

ASSESSMENT TOOL – 2

Course Name: Internet Programming

Course Code: CSA4305

Course Outcome Covered: CO-1

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

Total Marks: 20

Student Name: Mathusri P

Register Number: 192311363

Question 1. HTTP Response Analysis

Given HTTP Response Header

HTTP/1.1 200 OK

Date: Mon, 01 Apr 2026 10:00:00 GMT

Content-Type: text/html

Content-Length: -150

Connection: keep-alive

1 Identify incorrect or invalid fields in the header

- The invalid field in the response header is:
- Content-Length: -150
- The Content-Length value cannot be negative. It must be a non-negative integer representing the size of the response body in bytes.
- The Date field should also be checked. The date and weekday should match correctly. If the actual weekday does not match the date, the header should be corrected.
- Other fields such as Content-Type: text/html and Connection: keep-alive are valid.

2 Analyze the issue with Content-Length and explain its impact

- The Content-Length header tells the browser how many bytes of response data it should read.
- A negative value such as -150 is invalid because the size of a response body cannot be less than zero.

This may cause the following problems:

- The browser may reject the response.
- The browser may display an incomplete page.
- The connection may be closed unexpectedly.
- The browser may wait for data incorrectly.
- Different browsers may handle the response differently.

Therefore, the Content-Length value must be corrected to the actual size of the response body.

3 Determine whether the status code matches the response correctness

The status code is:

- HTTP/1.1 200 OK
- The 200 OK status code means that the request was successfully received, processed, and completed by the server.
- However, the response contains an invalid Content-Length value. Therefore, although the server reports success, the response header itself is not completely valid.
- The status code may still be 200 OK when the requested resource is successfully generated, but the header must be corrected before sending the response.
- If the server cannot generate a valid response due to an internal problem, a status code such as 500 Internal Server Error may be more appropriate.

1.4 Suggest corrections for the identified errors

The negative Content-Length value should be replaced with the actual number of bytes in the response body.

A corrected response header may look like this:

HTTP/1.1 200 OK

Date: Wed, 01 Apr 2026 10:00:00 GMT

Content-Type: text/html; charset=UTF-8

Content-Length: 150

Connection: keep-alive

Alternatively, the server can omit Content-Length and use chunked transfer encoding:

Transfer-Encoding: chunked

Conclusion

The response mainly contains an invalid negative Content-Length. Correcting it ensures that the browser can properly receive, interpret, and display the web page.

Question 2. DOM Structure Analysis

Given DOM Structure

```
<html>
<head>
  <title>TestPage</title>
</head>
<body>
  <div>
    <p>HelloWorld
  </div>
</body>
</html>
```

1 Identify missing or improperly closed elements

The `<p>` element is not properly closed.

The incorrect part is:

```
<p>HelloWorld
```

It should be written as:

```
<p>Hello World</p>
```

The `<div>` element is closed, but the paragraph inside it should be closed before the `</div>` tag.

The document also does not include:

- `<!DOCTYPE html>`
- The lang attribute in the `<html>` element.
- A character encoding declaration.

2 Analyze how the browser will interpret this structure

Modern browsers use error-recovery rules when they find invalid HTML.

The browser will usually automatically insert the missing closing `</p>` tag before the closing `</div>` tag.

The browser may internally interpret the code as:

```
<div>
  <p>HelloWorld</p>
```

</div>

Therefore, the text may still appear on the page even though the original HTML is invalid.

However, relying on automatic browser correction is not recommended because different browsers may create slightly different DOM structures.

3 Determine the impact on rendering and layout

The missing closing tag may cause:

- Unexpected spacing.
- Incorrect CSS styling.
- Improper element alignment.
- Problems when JavaScript accesses DOM elements.
- Different layout results across browsers.
- Difficulty in maintaining and debugging the code.

In a simple page, the text may display normally. In a complex page, the error may affect the position and styling of other elements.

2.4 Suggest corrections to make the DOM valid

The corrected HTML structure is:

<!DOCTYPE html>

<html lang="en">

<head>

 <meta charset="UTF-8">

 <title>Test Page</title>

</head>

<body>

 <div>

 <p>Hello World</p>

 </div>

</body>

</html>

This structure is valid because:

- It includes the HTML5 doctype.
- All elements are properly opened and closed.
- The language is specified.
- The character encoding is defined.
- The paragraph is correctly nested inside the <div>.

Conclusion

Although browsers can automatically repair some missing tags, developers should always write properly nested and valid HTML to ensure consistent rendering and reliable DOM behaviour.

3. GET vs POST in Login System

1 Interpret the differences in data transmission between GET and POST

GET Method

The GET method sends form data as part of the URL query string.

Example:

/login?user=admin&pass=1234

The username and password are visible in:

- Browser address bar.
- Browser history.
- Server logs.
- Proxy logs.
- Bookmarks.
- Shared links.

GET requests are mainly used to retrieve data from a server.

POST Method

The POST method sends data inside the HTTP request body.

Example request:

POST /login HTTP/1.1

Content-Type: application/x-www-form-urlencoded

user=admin&pass=1234

The data is not displayed in the browser address bar.

POST is generally used to submit sensitive information, create data, or perform actions that change the server state.

2 Analyze which method is more suitable for login and why

The POST method is more suitable for login systems.

Reasons include:

- Login credentials are not displayed in the URL.
- Credentials are not normally stored in browser history.
- The request is less likely to be bookmarked accidentally.
- POST supports larger amounts of data.
- It is suitable for submitting information to the server.
- It works securely when combined with HTTPS.

However, POST alone does not encrypt the data. HTTPS must be used to protect the credentials during transmission.

3 Identify security risks in using GET for authentication

Using GET for login creates several security risks:

- Passwords are visible in the address bar.
- Credentials may be stored in browser history.
- URLs may be saved in server logs.
- Credentials may appear in proxy and network logs.
- URLs may be copied and shared accidentally.
- Credentials may be exposed through the HTTP Referer header.
- Search engines or monitoring tools may record the URL.
- The page may be bookmarked with the credentials.

Therefore, GET should never be used to transmit passwords or authentication details.

4 Suggest best practices for secure login implementation

The following practices should be followed:

1. Use the POST method for login forms.
2. Use HTTPS to encrypt communication.
3. Store passwords using secure hashing algorithms such as Argon2 or bcrypt.
4. Never store passwords as plain text.

5. Validate inputs on both client and server sides.
6. Use secure session management.
7. Set cookies with Secure, HttpOnly, and SameSite attributes.
8. Implement protection against SQL injection.
9. Use prepared statements or parameterized queries.
10. Add protection against Cross-Site Request Forgery.
11. Limit repeated failed login attempts.
12. Use multi-factor authentication when required.
13. Display general error messages such as “Invalid username or password.”
14. Do not expose whether a particular username exists.

Correct Login Form

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Secure Login</title>
</head>
<body>

  <form action="/login" method="POST">
    <label for="username">Username:</label>
    <input
      type="text"
      id="username"
      name="username"
      autocomplete="username"
      required
    >

    <br><br>

    <label for="password">Password:</label>
```

```
<input
  type="password"
  id="password"
  name="password"
  autocomplete="current-password"
  required
>

<br><br>

<button type="submit">Login</button>
</form>
```

```
</body>
```

```
</html>
```

Conclusion

POST is more appropriate than GET for login operations. However, secure login requires POST, HTTPS, password hashing, input validation, and proper session management.

4. HTML Code Inspection

Given HTML Code

```
<!DOCTYPE html>

<html>

<head>

  <title>Sample</title>

</head>

<body>

  <h1>Welcome

  <p>This is a paragraph

  

</body>

</html>
```


1 Identify missing closing tags

The following closing tags are missing:

- `</h1>`
- `</p>`

The heading should be written as:

```
<h1>Welcome</h1>
```

The paragraph should be written as:

```
<p>This is a paragraph</p>
```

The `<img>` element is a void element in HTML5. Therefore, it does not require a closing tag.

2 Analyze issues with the `<img>` tag

The image element is:

```

```

The main issue is the absence of the alt attribute.

The alt attribute provides alternative text when:

- The image cannot be loaded.
- A screen reader reads the page.
- The user has disabled images.
- The file path is incorrect.

A better version is:

```

```

The developer may also specify image dimensions to reduce layout movement:

```

```

3 Determine how browsers will handle these errors

Modern browsers try to correct invalid HTML automatically.

The browser may:

- Automatically close the `<h1>` element.
- Automatically close the `<p>` element.

- Display the image if the path is correct.
- Construct a corrected DOM internally.
- Apply unexpected heading or paragraph styles.
- Produce inconsistent layouts in different browsers.

For ex: text may appear larger than expected if the browser treats paragraph content as part of the heading.

4 Suggest a corrected version of the code

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta
    name="viewport"
    content="width=device-width, initial-scale=1.0"
  >

  <title>Sample</title>
</head>

<body>

  <main>

    <h1>Welcome</h1>

    <p>This is a paragraph.</p>

    

  </main>

</body>

</html>
```

Conclusion

The corrected code properly closes the heading and paragraph elements and adds alternative text to the image. This improves standards compliance, accessibility, and cross-browser consistency.

5. Browser-Server Communication Flow

Given Communication Sequence

1. User enters a URL in the browser.
2. DNS lookup occurs.
3. HTTP request is sent.
4. Server processes the request.
5. Response is returned.
6. Browser renders the page.

1 Interpret each step in the communication flow

Step 1: User Enters a URL

The user types a web address into the browser.

Example:

`https://www.example.com`

The browser analyzes the URL to identify:

- Protocol, such as HTTPS.
- Domain name, such as `www.example.com`.
- Port number.
- Path of the requested resource.

Step 2: DNS Lookup Occurs

The browser needs the server's IP address before connecting to it.

The Domain Name System converts the domain name into an IP address.

Example:

`www.example.com` → `93.184.216.34`

The browser first checks:

- Browser DNS cache.
- Operating system cache.
- Router cache.
- Internet Service Provider DNS server.
- Authoritative DNS server.

Step 3: HTTP Request Is Sent

After finding the IP address, the browser connects to the server.

For HTTPS, the browser also creates a secure TLS connection.

The browser then sends an HTTP request.

Example:

GET / HTTP/1.1

Host: www.example.com

User-Agent: Browser

Accept: text/html

Step 4: Server Processes the Request

The server receives the request and determines what resource is needed.

The server may:

- Locate an HTML file.
- Run server-side code.
- Access a database.
- Verify authentication.
- Process user input.
- Generate a dynamic response.

Step 5: Response Is Returned

The server sends an HTTP response containing:

- Status code.
- Response headers.
- Response body.

Example:

HTTP/1.1 200 OK

Content-Type: text/html

Content-Length: 2500

The response body contains the requested HTML page.

Step 6: Browser Renders the Page

The browser reads the HTML and constructs the Document Object Model.

It then:

- Downloads CSS files.
- Downloads JavaScript files.

- Downloads images and fonts.
- Creates the CSS Object Model.
- Executes JavaScript.
- Calculates the layout.
- Paints the content on the screen.

The final web page is then displayed to the user.

2 Identify where delays can occur

Delays can occur during:

DNS Lookup

A slow DNS server can increase loading time.

Network Connection

Long physical distance, poor internet speed, and network congestion can cause delays.

TLS Handshake

Creating an encrypted HTTPS connection requires additional communication between the browser and server.

Server Processing

The server may take more time because of:

- Complex application logic.
- Heavy traffic.
- Slow database queries.
- Insufficient hardware resources.
- External API calls.

Large Response Size

Large HTML, CSS, JavaScript, image, video, or font files require more download time.

Browser Rendering

Complex CSS, excessive JavaScript, and large DOM structures can slow down page rendering.

3 Analyze the role of DNS in the process

DNS acts like the telephone directory of the Internet.

Users remember domain names such as:

www.example.com

Computers communicate using IP addresses such as:

93.184.216.34

DNS translates the domain name into the correct IP address.

Without DNS, users would need to remember the numerical IP address of every website.

DNS is important because it:

- Makes websites easier to access.
- Maps domain names to servers.
- Supports multiple servers for one website.
- Helps distribute traffic.
- Supports failover and load balancing.
- Allows servers to change IP addresses without changing the domain name.

4 Suggest optimizations to improve performance

The following techniques can improve browser-server communication:

1. Use DNS caching.
2. Choose a fast and reliable DNS provider.
3. Use a Content Delivery Network.
4. Enable browser caching.
5. Compress files using Gzip or Brotli.
6. Minify HTML, CSS, and JavaScript.
7. Optimize and compress images.
8. Use modern image formats such as WebP or AVIF.
9. Reduce unnecessary HTTP requests.
10. Use HTTP/2 or HTTP/3.
11. Enable persistent connections.
12. Optimize database queries.
13. Use server-side caching.
14. Load JavaScript using defer where appropriate.
15. Use lazy loading for images.
16. Reduce the size of the DOM.
17. Place servers closer to users through edge locations.
18. Avoid unnecessary redirects.

Conclusion

Browser-server communication consists of URL interpretation, DNS resolution, request transmission, server processing, response delivery, and browser rendering. Performance can be improved by optimizing the network, server, files, database, caching, and rendering process.

Overall Conclusion

This assessment analyzes important Internet Programming concepts such as HTTP headers, DOM structure, GET and POST methods, valid HTML coding, and browser-server communication.

Correct HTML structure ensures consistent rendering and accessibility. Proper HTTP headers help browsers correctly interpret server responses. POST with HTTPS is suitable for secure login systems. DNS helps locate servers, while caching, compression, and optimized resources improve website performance.

Code of Conduct

I, Mathusri P-192311363 (**Name / Register Number**), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Mathusri P